

**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

SC4020 Data Analytics and Mining:

Project 2

Name	Matric Number	Contributions
Prakritipong Phuvajakrt	U2222277F	Task 2 + report + video
Trisha Bankata Mishra	U2222358J	Task 1 + report + video
Neha Ramesh	U2023864F	Task 2 + report + video
Shubhangam Prasad	U2129974E	Task 1 + report + video
Bhati Nancy	U2222368G	Task 3 + report + video

Task 1: Analysis of Co-occurrence Patterns of Points of Interest (POI)

Data Processing

Each dataset was reorganised to assign distinct (x, y) coordinates with a list of POI categories for each grid cell, forming a unique identifier (`unique_grid_id`). This allowed us to bundle the categories that occurred together within each grid cell. The data was then one-hot encoded to prepare for frequent pattern mining, to represent each POI category in a binary format. This step was imperative for applying the Apriori algorithm effectively.

Apriori Algorithm

Frequent itemsets were computed using the Apriori algorithm, implemented through the `mlxtend` library. The one-hot encoded dataset was used, and the minimum support threshold was set, balancing performance and memory usage. A lower support threshold could result in many infrequent or noisy patterns being considered, while a higher threshold might overlook potentially important itemsets.

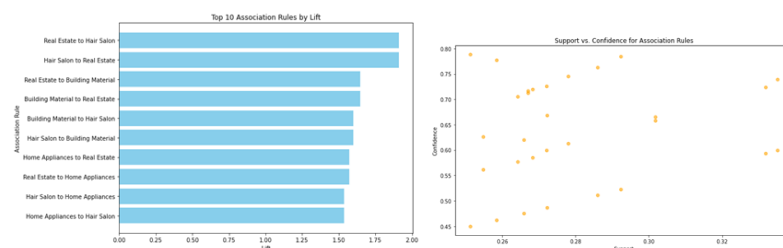
For Cities A, and C a minimum support threshold of 0.3 was used. For cities B and D, a minimum support of 0.1 was used. Using a lower threshold was resulting in noisier patterns being detected, which led to inconclusive results. We had to adjust the threshold to get the best values possible for each city.

After generating frequent itemsets, the dataset was filtered to include only itemsets of length greater than 1, to focus on combinations of categories rather than individual POIs. The generated itemsets were then processed using the `association_rules` function, which allowed us to evaluate rules based on metrics like lift and confidence. Lift was preferred over confidence, as it normalises the confidence and provides more context. The resulting rules provided insights into which POI categories tend to co-occur across different parts of each city.

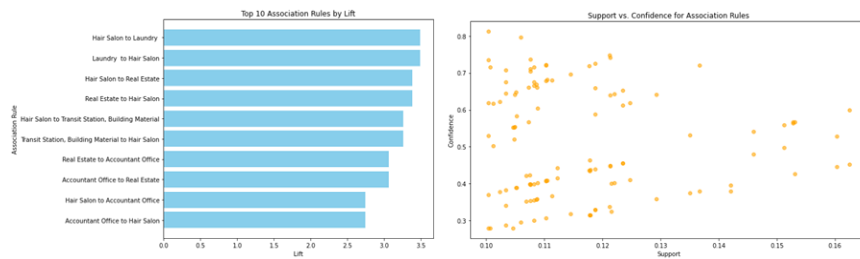
Analysis of Co-occurrence Patterns

The analysis of frequent itemsets revealed interesting variations between the cities:

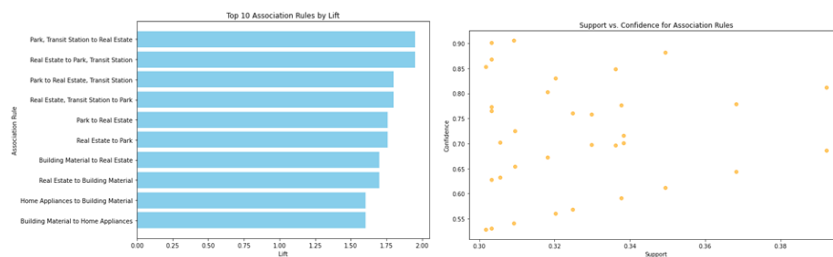
- City A: City A showed moderate clustering of POI categories, with several combinations having notable support levels.



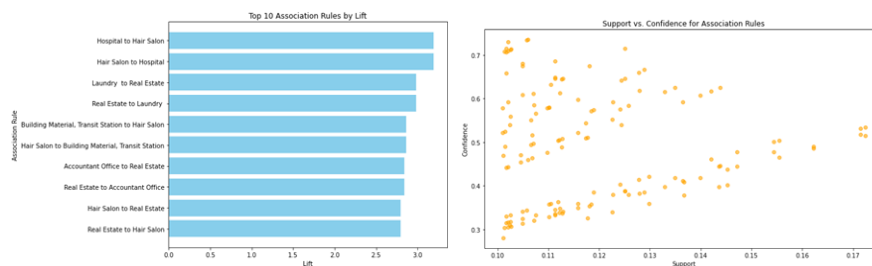
- City B: City B had lower support values overall, indicating a more dispersed distribution of POIs. This suggests less clustering and possibly a more spread-out urban layout.



- City C: City C exhibited the highest overall support levels, suggesting denser clustering of certain POI categories compared to the other cities. This indicates a more compact urban layout with specific areas having multiple related POIs.



- City D: City D showed a pattern similar to City A but with some unique category combinations that did not appear prominently in other cities.



These findings provide a foundation for understanding how different types of amenities are distributed across urban environments and how urban planning decisions influence the clustering of Points of Interest.

Task 2: Mining Sequential Patterns

Triplets generation

The first 10 days of dataset A (due to large running time, we opted to use a smaller subset) and 30 days of datasets B, C and D were taken and modified into a form compatible with the trackintel library, i.e., the 'd' and 't' columns were changed into the datetime format and the other columns were renamed.

Coordinates in each dataset are processed to generate staypoints and triplegs. Staypoints are generated on the preprocessed data for each city, using a sliding window method with configurable thresholds for distance, time, and gap, ensuring that only relevant data is captured. To generate a tripleg, you examine the sequence of staypoints and treat the movement from one staypoint to the next as a separate segment. Each tripleg corresponds to the travel from one staypoint to another.

GSP Algorithm

Before implementing the GSP algorithm on the triplegs, we first preprocess the triplegs by converting the start and end times to datetime formats for more convenient data handling. Then, we set a cutoff date to be 30 days from the starting date of the dataset to filter only data from the first 30 days allowing the code to be more configurable. The 'geom' column originally contains linestring data which can be difficult to work with for coordinate calculations and frequent sequence mining, hence we parsed them into tuples of integers. The triplegs are then grouped by user_id and flattened to convert them back into a single list of coordinate tuples.

The triplegs are then split into smaller sizes for more convenience. In this instance, we decided to split long triplegs into sub-triplegs of max length 500.

In the GSP algorithm,

- 1) Generate initial candidate subsequences using **generate_candidates(sequences, length)** function starting from length=2.
- 2) Count support of each candidate subsequence using **count_support(candidates, sequences)** to check if the candidate subsequence satisfies the conditions of support $\geq$ minsup. Each candidate is also checked if it is a subsequence of any other sequences using **is_subsequence(candidate, sequence)**.
- 3) For candidates that are infrequent, we eliminate them using **prune_candidates(support_count, min_support)** then only keeping a dictionary of candidates that meet the minsup threshold. They are then stored in the **all_frequent_sequences** list.
- 4) Frequent subsequences are then merged to form candidate subsequences of longer length by incrementing **length += 1** and generated using **generate_candidates(sequences, length)** again.
- 5) The process is then repeated until all frequent sequences are generated.

The GSP algorithm is used on triplegs of datasets B,C and D. However, when implemented on dataset A, the computation time was significantly longer due to the much larger size of dataset A.

For dataset A, an alternative GSP algorithm is used to parallelise the support counting process by splitting sequences into chunks using **multiprocessing** library instead of sequentially like in the original GSP code. The num_workers parameter indicates the number of CPU cores used to parallelise the processes where each worker counts the support independently and combines the results at the end. By utilising multiple CPU cores at once, computation speed is increased to handle large data size.

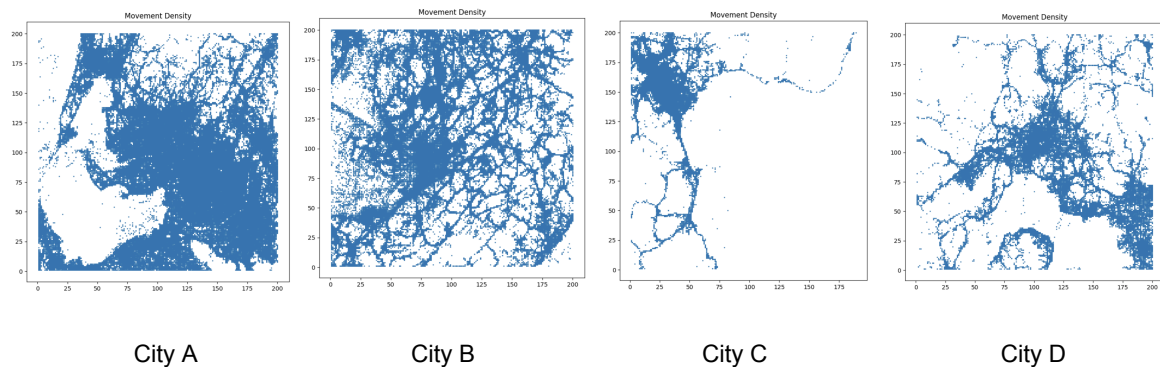
Minsup threshold was also increased to 0.1% of the triplegs dataset size compared to 0.01% used for B,C and D because we want to prune off more candidates to reduce computation. This may sacrifice some sequences that could be considered frequent in other cases in exchange for faster computations and also helps us pinpoint sequences that are more statistically relevant as using lower support may introduce more noise in our data. We also just sampled the data by only including 500 user ids due to time limitations.

Analysis

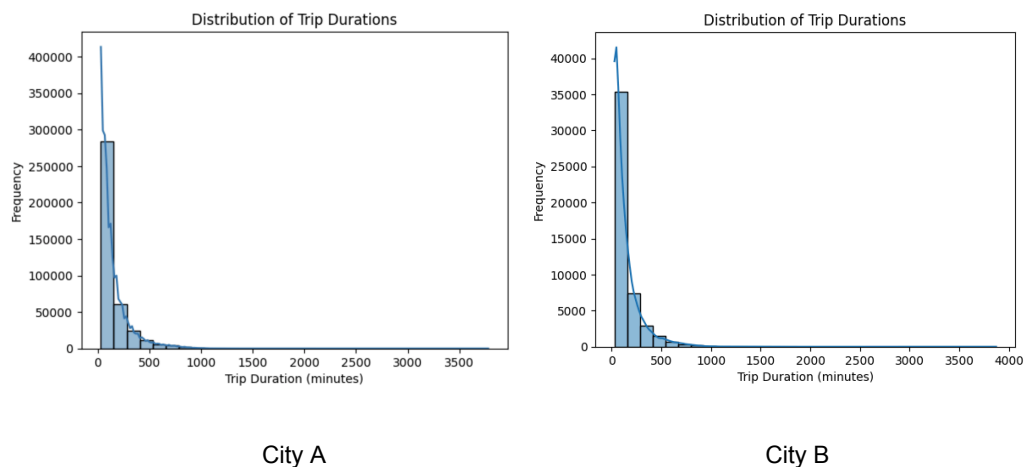
1. Triplegs

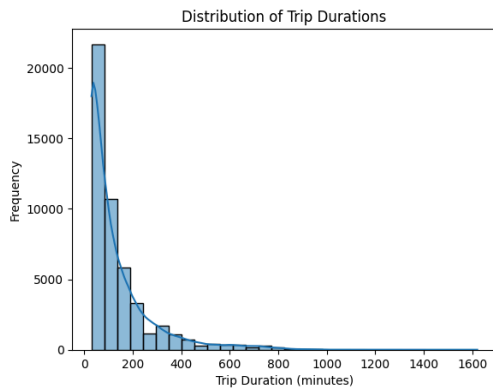
1.1. Movement Density

Point is used to effectively flatten the list of sequences (coordinates) into a list of Point objects which is then converted into a GeoDataFrame using geopandas and plotted as shown. We can see the density of the movement patterns in each city.

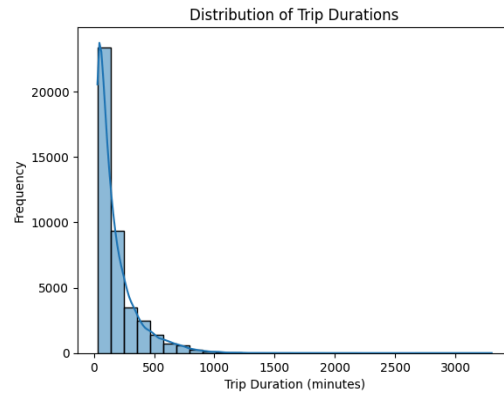


1.2. Distribution of trip duration



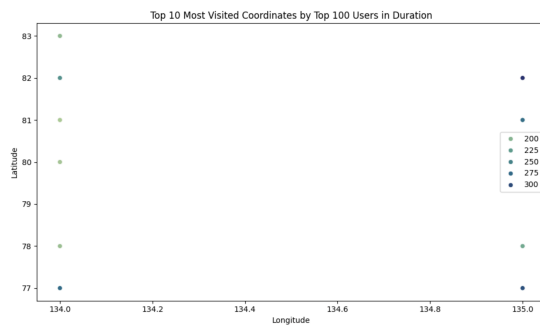


City C

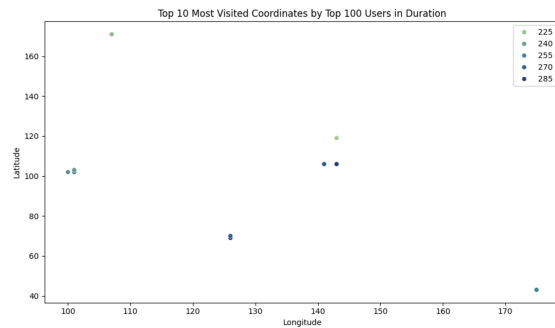


City D

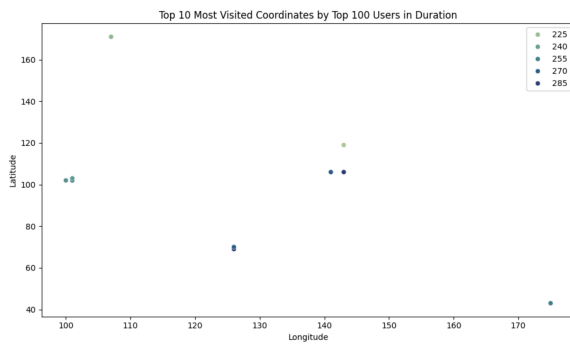
1.3. Top 10 Most Visited Coordinates by Top 100 Users in Duration



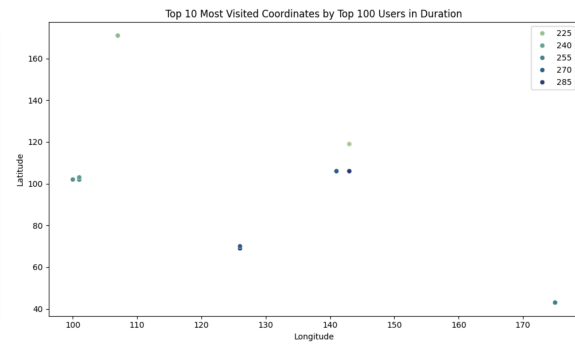
City A



City B



City C

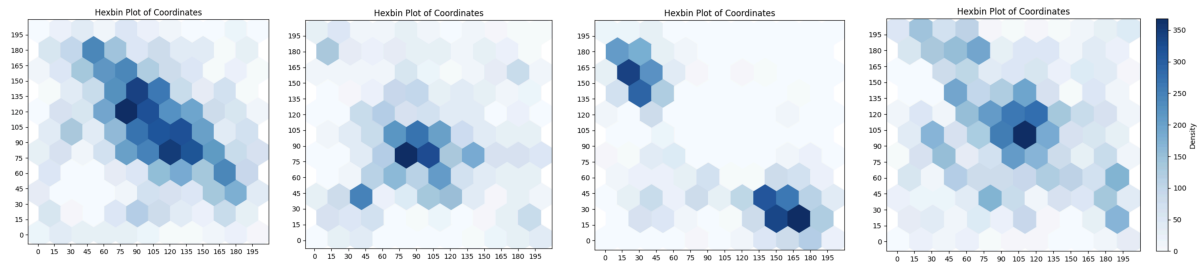


City D

2. Frequent Sequences

2.1. Hexbin of top coordinates in frequent sequences.

As this shows the concentration of frequent coordinates, we can assume darker hexes show crowded spots in the city.



City A

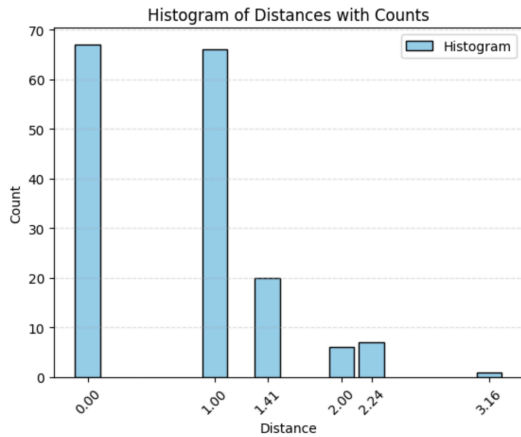
City B

City C

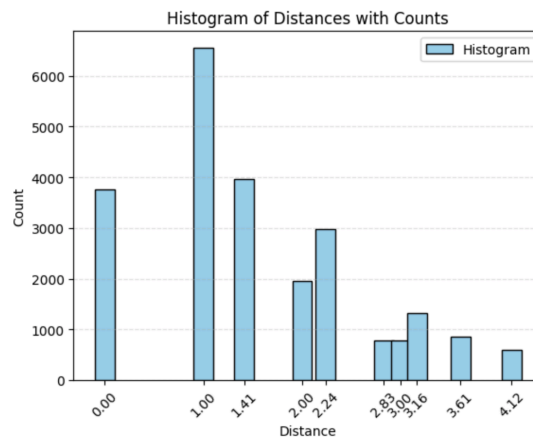
City D

2.2. Histogram of Distance between First and Last Coordinate in a Preferred Sequence against Counts

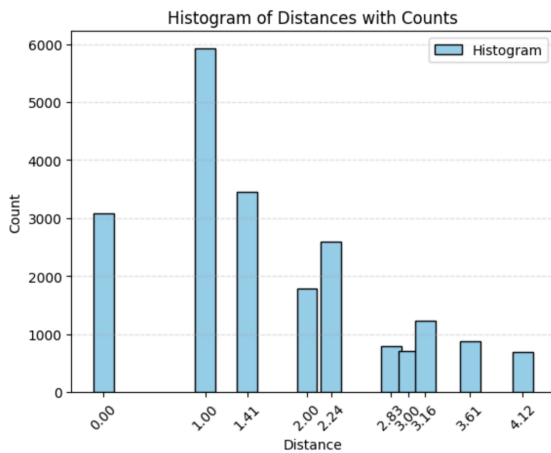
These distances are calculated using Pythagoras theorem. By taking the squares of these distances, and logically assigning possible squares of whole numbers that could sum up to the value shown, we see that most of these movements are only one or two coordinates apart on x and/or y.



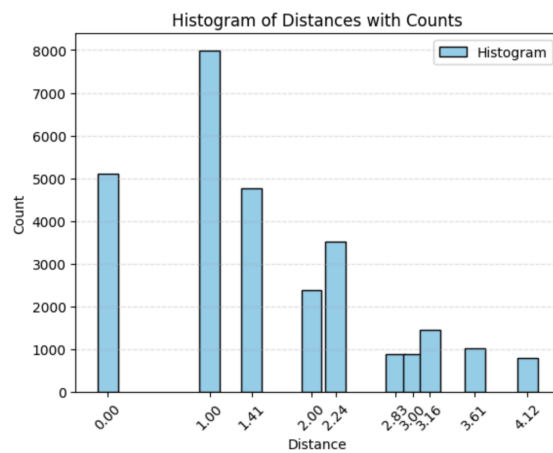
City A



City B



City C



City D

Task 3: Open Advanced Tasks

Application- Travel Based Recommendation System

The aim of this application is to provide personalised destination suggestions to users based on their past interactions and preferences. This system uses Collaborative Filtering (CF) methodologies to recommend Points of Interests (POIs) across different cities.

Data Processing

1. Datasets were consolidated into a unified framework using Dask for large scale data handling.
2. Features such as user IDs (uid), POI categories and geographical coordinates (x,y) were normalised to ensure consistency.
3. A user-item interaction matrix is created by aggregating user activities across different POI categories.

Methodology

Collaborative Filtering(CF) is used to identify patterns and similarities among users or items (POIs) to generate relevant suggestions.

1. Items-Based Collaborative Filtering (IBCF):
 - Leveraged the similarity between POI categories to recommend destinations.
 - Calculated Cosine Similarity between POI categories to identify relationships.
 - Used a K-Nearest Neighbors (KNN) algorithm to identify most similar items.
2. User-Based Collaborative Filtering (UBCF):
 - Incorporated similarities between users to suggest POIs that similar users have interacted with.
 - Predictions were generated by aggregating ratings or interaction scores based on the nearest neighbours.

Algorithms

1. K-Nearest Neighbors (KNN)
 - Identified top N most similar items or users.
 - Incorporated into both User-Based and Item-Based Collaborative Filtering approaches
2. Dask Framework
 - Efficient handling of large-scale datasets for preprocessing and merging tasks

Evaluation

1. To assess the model performance, the Root Mean Squared Error (RMSE) metric was applied to compare the predicted ratings against actual ratings from the test data.
2. The lower the RMSE, the more accurate the recommendation System.
3. An RMSE value of 0.2646 was obtained.

Root Mean Squared Error (RMSE): 0.2646

Further Improvement

Algorithms such as content-based filtering, Apriori Algorithm or Hybrid Filtering (Combining collaborative and content-based filtering) can be used for better accuracy.